

Mini-Agent-RAG 文本处理技术路线说明

一、文档定位

本文只总结当前项目在文本知识处理、文本检索、文本生成方面的技术路线。

文档聚焦本地知识库问答主链路，只讨论文本处理相关能力。

目标是用更短、更直接的方式说明：项目如何从文本中找证据、组织证据，并基于证据回答问题。

二、项目目标

Mini-Agent-RAG 面向高校文档问答场景，依托本地知识库完成检索增强问答，也就是常说的 RAG（检索增强生成）。

它的核心任务不是开放式闲聊，而是围绕用户问题，从文本资料中找到相关证据，整理成可用上下文，再生成尽量准确、可追溯的回答。

三、文本处理主链路

当前项目的文本主链路可以概括为：

1. 文本入库
2. 文本切分
3. 查询改写
4. 混合检索
5. 启发式重排
6. 模型重排
7. 上下文组装
8. 生成回答
9. 完备性与事实审校
10. 输出答案与引用

各步骤的当前做法如下：

3.1 文本入库

项目支持常见文本资料进入本地知识库。

入库后会保留标题、来源、分段信息等基础元数据，为后续检索和排序提供线索。

3.2 文本切分

文档不会整篇直接送入检索，而是先切成较小文本块。

项目同时保留句级线索，便于在命中小块后继续向更完整的上下文扩展，减少“命中了但看不到完整答案”的情况。

这类做法接近 `Small-to-Big` 思路，也就是先命中小片段，再回到更完整的大段上下文回答。

3.3 查询改写

用户问题进入系统后，会先做查询改写和多查询扩展。

当原问题过于口语化、过短或表达不完整时，系统会补充更适合检索的表达，必要时还会生成一段假设性检索文本，用来增强召回覆盖。

这里对应的核心做法包括 `Multi-Query` 和 `HyDE`。

3.4 混合检索

项目不是只走单一路径，而是把稠密检索和关键词检索结合起来。

这样既能保留语义相近的召回能力，也能保留对专有名词、时间词、固定表述的匹配能力，再通过融合排序得到更稳的候选结果。

这一步可以概括为 `Hybrid Retrieval`。

3.5 启发式重排

初步召回后，系统会先用规则做一轮重排。

重点考虑问题和文本块的词项重合、答案窗口命中、标题线索、时间线索等因素，让更像答案承载块的内容排到前面。

这一步属于 `Heuristic Rerank`。

3.6 模型重排

在规则排序基础上，项目继续使用重排模型做精排。

这一层的作用是进一步判断“问题和候选文本是否真的匹配”，减少只靠关键词命中带来的误排。

这一步属于 `Model Rerank`。

3.7 上下文组装

排到前面的证据不会原样堆给模型，而是要做整理。

系统会控制上下文长度，去掉重复块，并尽量保留关键句、关键段和跨段补充信息，使最终送入模型的内容更完整、更可回答。

这一层可以理解为 `Context Assembly`。

3.8 生成回答

在整理后的上下文基础上，模型生成最终回答。

回答阶段强调先回答结论，再补充必要说明，尽量避免空泛复述。

3.9 完备性与事实审校

项目在生成后还会做收口。

一方面检查是否漏答并列子问题，另一方面检查是否出现证据未支持的扩写、时间外推或主语错配。

如果证据不足，系统倾向于保守回答，而不是强行补全。

3.10 输出答案与引用

最终结果不仅返回答案，也返回引用证据。

这样可以让用户看到答案来自哪些文本材料，便于追溯和核对。

四、核心技术路线

4.1 查询理解

项目会尽量识别不同类型的文本问题，重点包括：

- 口语化问题
- 比较题
- 时间敏感题
- 多文档整合题

这一步的意义是让后续检索、排序和回答方式更贴近问题本身。

4.2 检索融合

项目采用多路召回后融合的方式，而不是依赖单一路径。

这样做的核心目的是提升召回覆盖，降低“答案就在库里但没被找到”的概率。

这条路线本质上就是 Hybrid Retrieval 加融合排序。

4.3 排序优化

系统先用启发式规则打分，再用模型做精排。

这条路线的重点不是“多拿一些结果”，而是把真正接近答案的证据尽量提前。

对应的两层分别是 Heuristic Rerank 和 Model Rerank。

4.4 上下文压缩与组织

项目的一个关键点不只是检索，而是把检索结果整理成可回答上下文。

如果上下文压缩过强，即使证据已经命中，模型也可能答不完整；因此项目持续在去重、片段长度和跨段组织上做优化。

4.5 生成收口

生成阶段不是简单把检索结果复述出来。

项目会重点约束三类问题：

- 并列问题不能漏项
- 证据不足时不能过度外推
- 时间敏感问题要保留具体时间条件